CANONICAL COMPANION DOCUMENT

BOX-NATIVE SOFTWARE DEVELOPMENT

GitHub for Boxes - Product Vision Supplement V4

A research-backed implementation ladder for moving software development from repository navigation to verified Boxes, while keeping Git as the durable version-control substrate.

V4 change: the product thesis is now tied to evidence. Every major rung is mapped to existing engineering mechanisms, academic precedents, reusable tools, explicit limits, and a validation gate.

Canonical relationship

This document supersedes Product Vision Supplement V3. It is the product-facing companion to the 35-page Private Engineering Delegation research proposal (2 October 2026), which remains the deeper research foundation for controlled disclosure, private transfer, verification and later proprietary execution [R1].

North star

A product manager, engineer or domain owner describes desired behavior. The system grounds that intent in the current software, proposes a reviewable Box, and gives the Box to a human, an agent, or both. The executor works against the Box rather than an undifferentiated repository. The resulting attempt is evaluated and, when accepted, becomes normal Git history.

Git remains underneath. The Box becomes the primary unit of intent, execution, evaluation and coordination.

Evidence key used throughout V4

Label	Meaning
ENGINEERING FOUNDATION	A current mechanism or tool already exists for this part.
RESEARCH PRECEDENT	Published work supports a narrower component or pattern.
PRODUCT DESIGN	Our proposed way of composing those pieces into Box-native development.
OPEN RESEARCH	The capability is not established and must be validated before it is claimed.

01 | PRODUCT + RESEARCH

The Box is a versioned work object, not a prompt

V3 defined the Box as intent + human-validatable behavior + context + edit boundary + environment + evaluator + Git mapping. V4 keeps that definition, but anchors each field to mechanisms that can exist independently of a language model.

Box field	Concrete representation	Why it is not “just AI”
Identity + version	Box ID, base commit, manifest hash, evaluator version	Deterministic identity and replay.
Intent	Decision record, goals, exclusions, evidence links	Requester approval can be stored and versioned.
Expected behavior	Examples, preview, state transitions, API behavior	Human oracle / review artifact.
Context	Files, symbols, interfaces, fixtures, docs, dependency closure	Can be conservative at first; does not require perfect slicing.
Edit boundary	Allowed paths/symbols/dependencies	Enforceable by workspace policy and patch checks.
Environment	Dev Container or equivalent pinned environment	Reproducible execution substrate [W4].
Evaluator	Tests, properties, contract checks, private checks	Machine acceptance independent of the solver [W5].
Git mapping	Base revision, attempt diff, merge target, provenance	Normal source control remains authoritative [W2][W20].

Research implication

The 35-page proposal already calls the structured Task Contract the central representation rather than a prompt, and requires traceability from approved intent to acceptance [R1, pp. 6-7]. V4 therefore treats “Task Contract” as the formal internal backbone of the broader “Box” product primitive.

What is actually novel

None of the individual mechanisms above is new. The thesis is the composition and the human interface: software work is created, navigated, assigned, executed and reviewed as Boxes; repositories and files become implementation substrate rather than the primary work abstraction.

Novelty should be claimed at the system/workflow level, not by pretending Git, containers, tests, contracts or remote IDEs are new.

02 | PRODUCT + RESEARCH

One Box platform, two surfaces, one API

The web and IDE are product surfaces over the same Box object. The research foundation suggests reusing existing workspace infrastructure first rather than inventing a new editor before the Box workflow is validated [R1, p. 22].

Surface	Primary job	Implementation reality
Box Web - control plane	Create, approve, browse, assign, discuss, review, govern and inspect Boxes.	Normal web application over Box/Git/evaluator services.
Box Workbench / IDE - execution plane	Open one Box, inspect behavior, edit code, run evaluator, use agent, review diff.	Initially existing browser/remote IDE infrastructure; later a dedicated Box-native IDE if earned.
Agent API	Consume the same Box contract and return an attempt.	Headless execution path; no UI dependency.
Git substrate	Store repository history, diffs, merge results and base revisions.	Worktrees can isolate concurrent Box attempts; sparse checkout can reduce visible files [W2][W3].

Precedent and current infrastructure

Codeon showed that IDE context capture and task handoff can reduce coordination overhead for remote assistance, but it did not solve automatic task compilation or confidentiality [W7]. Coder and VS Code Remote Development show that humans and coding agents can already work inside centrally controlled, reproducible remote workspaces without requiring source code to live on a local machine [W9][W10].

Product design decision

Do not build “VS Code again.” Build the Box model, Box control plane and Box protocol first. Embed or integrate an existing editor/workspace stack. A custom Box IDE becomes justified only when Box-specific navigation and execution prove valuable.

What the Box-native IDE eventually changes

- Primary navigation: Boxes and Box dependencies, not the repository file tree.
- Persistent expected-behavior pane beside implementation and evaluator evidence.
- First-class “work myself / solve with agent / hybrid” execution over the same Box.
- Box-aware Git history: attempt -> evaluation -> review -> commit/merge provenance.

03 | PRODUCT + RESEARCH

Git under the hood: a concrete Box-to-commit lifecycle

The Box does not replace Git. It adds a higher-level work identity that can be mapped deterministically onto ordinary source-control operations.

Step	Box operation	Concrete Git / workspace mechanism
1	Create Box v1 against a known repository state	Record base commit SHA and Box manifest hash.
2	Open isolated attempt	Create git worktree or isolated clone at the base revision [W2].
3	Project context	Optionally use sparse-checkout for convenience/performance; never mistake it for a security boundary [W3].
4	Execute	Human/agent edits within the workspace; evaluator runs against pinned environment.
5	Submit attempt	Capture patch/diff + evaluator version + artifact hashes.
6	Review and merge	Normal code review/merge semantics; resolve drift and conflicts.
7	Record provenance	Commit trailer/metadata can carry Box-ID / Attempt-ID; richer provenance remains in Box service [W19][W20].

Why worktrees are a strong MVP primitive

Git explicitly supports multiple working trees attached to one repository, each associated with its own checked-out state and metadata [W2]. That is very close to the operational need for concurrent Box attempts. The Box service can manage worktrees automatically so the user experiences “open Box,” not “manage branches.”

Important limit

Sparse-checkout only changes which tracked files are present in the working tree. Git documentation describes it as working with a subset of files; it is a convenience/performance mechanism, not confidentiality [W3]. Restricted proprietary Boxes therefore require a different isolation and release model in later stages.

MVP rule: preserve ordinary Git semantics whenever possible. Box metadata should augment source control, not fork the project into a proprietary VCS.

04 | PRODUCT + RESEARCH

Product ladder: validate the primitive before the hard research

The roadmap is deliberately staged. Each rung must be useful on its own and produce evidence for the next. The ordering also reconciles the product vision with the research proposal's warning not to begin with a new IDE, universal codebase support, or an open marketplace [R1, p. 22].

Stage	Product delivered	Main evidence gate
0 - Box nucleus	Box schema, intent approval, evaluator, attempt model, Git mapping.	Can one task be represented, replayed, solved and merged reproducibly?
1 - Open Box MVP	Trusted/non-sensitive repos; full repo allowed when slicing is uncertain.	Do users prefer intent -> Box -> evaluate -> merge over repo-first task setup?
2 - Box Web + Workbench	Web control plane plus embedded/existing remote editor execution.	Does the two-surface model reduce handoff/context friction?
3 - Box-native team product	Assignments, Box graph, agent execution, review, Box history, merge queue.	Can teams run meaningful internal work primarily through Boxes?
4 - Restricted proprietary Box	Authorized context, isolated execution, private acceptance, controlled transfer.	Does the boundary reduce exposure while preserving independent completion?
5 - Semantically transformed Box	Extract/parameterize/simulate/contract selected private responsibilities.	For supported classes, does external contract satisfaction imply private fit?
6 - External Box network	Route frozen, independently executable Boxes to vetted external executors.	Does external execution save total effort without breaking acceptance/security?
7 - Optional bounty market	Reward, reservation, identity, dispute and payment policy.	Does a market improve routing/liquidity after Box preparation already works?

The first product does not need to solve confidentiality. The end product eventually does.

05 | PRODUCT + RESEARCH

Stage 0-1 research: intent, expected behavior and evaluator

What existing evidence supports

Conversational requirements elicitation is plausible but not solved. ReqElicitGym provides 101 website scenarios and reports that evaluated models elicited fewer than half of implicit requirements, which argues for reviewable previews, explicit unknowns and requester approval rather than treating fluent AI output as truth [W6]. The test-oracle literature cited in the research proposal separately warns that generating tests and knowing the correct expected outcome are different problems [R1, p. 26].

What to build

- Intent conversation grounded in repo state, existing behavior, tests, docs and explicit user evidence.
- A reviewable expected-behavior artifact: examples, before/after flow, API interactions, state table or UI preview.
- A deterministic Box manifest with base commit, editable surface, environment and evaluator.
- Evaluator split: solver-visible checks for fast feedback; held-out or independently authored checks for stronger acceptance where appropriate.

Evaluator precedent

SWE-bench demonstrates a practical issue-to-patch evaluation harness: reproducible Docker environments, patch application, test execution and grading [W5]. It is not enough by itself for product correctness, but it proves the mechanics of reproducible task instances and patch evaluation are ordinary engineering rather than speculative infrastructure.

Stage 1 experiment

Measure	Minimum useful question
Intent alignment	Did the requester understand and approve the expected behavior before implementation?
Box setup effort	How long from informal request to runnable Box? Compare manual baseline.
No-missing-context rate	Can a fresh human/agent start without asking for repo-navigation help?
Evaluator validity	How often do passing attempts fail independent review or broader tests?
Integration success	How often does an accepted attempt apply cleanly to the pinned base and merge?
Preference	Would users choose the Box workflow again for the same task class?

OPEN RESEARCH: “AI understands the PM” is not an assumption. The product should expose uncertainty and make outcome approval a first-class gate.

06 | PRODUCT + RESEARCH

Stage 2 research: Web + Workbench without inventing infrastructure

Reusable execution substrate

The Dev Container specification is an open standard for enriching containers with development-specific tooling and settings [W4]. Coder provides centrally managed developer/agent workspaces and supports browser IDEs, VS Code, JetBrains and agent execution [W9]. VS Code Remote Development can run the editor against containers or remote machines without requiring source code on the local machine [W10].

A practical first implementation

Layer	Build vs reuse	Suggested first approach
Box Web	Build	Box creation, behavior approval, status, attempts, evaluator results, assignments.
Workspace provisioner	Reuse	Dev Container + Docker locally, or Coder/another remote workspace platform in cloud.
Editor	Reuse first	VS Code/code-server/Monaco shell; add Box-specific side panels and commands.
Terminal/debugger	Reuse	Existing editor/workspace capabilities.
Agent execution	Integrate	Same Box manifest delivered to a headless agent inside the workspace.
Box protocol	Build	Open Box, get context, run evaluator, submit attempt, stream status.
Git orchestration	Build thin layer	Worktree/branch/diff/merge helpers behind Box actions.

Research precedent for handoff

Codeon captured relevant IDE context for an asynchronous remote helper and reintegrated responses into the developer's environment; its study reported materially better task completion than the comparison tools [W7]. CrowdMicroservices used preconfigured environments and behavior-driven microtasks to implement a small service across 350 microtasks, providing a narrow precedent for self-contained programming packages [W8]. Neither establishes the full Box platform, but both reduce the "sandcastle" risk around context-rich handoff and runnable microtasks.

Stage 2 success is a workflow result, not an editor result: the user should feel that "opening a Box" gives an immediately actionable workspace.

07 | PRODUCT + RESEARCH

Stage 3 research: Boxes as the team operating model

What changes when Boxes become primary

A team product adds dependencies, assignments, attempts, merge ordering, provenance and agent/human routing. Git remains authoritative for code; Box history becomes authoritative for why the change existed, which behavior was approved, which evaluator version ran, and how an accepted attempt was produced.

Capability	Grounding	Open question
Concurrent Box attempts	Git worktrees/branches already isolate work [W2].	How should attempts share caches and reconcile drift?
Reproducible evaluator	Containerized evaluation is established [W5].	How much task-specific evaluator generation is reliable?
Environment consistency	Dev Containers / managed workspaces [W4] [W9].	What is the cold-start cost across stacks?
Agent + human same workspace	Coder and modern IDE infrastructure support both [W9].	Does one Box contract serve both equally well?
Provenance	SLSA defines provenance concepts; Git trailers can carry metadata [W19][W20].	Which Box evidence belongs in Git vs platform DB?
Task dependencies	Normal DAG/merge-queue engineering patterns.	How to detect semantic conflicts beyond textual merge?

What V4 would test before “dedicated Box IDE”

- Do engineers spend materially less time locating relevant code and reconstructing intent?
- Do agents need fewer context-recovery tool calls or fewer broad repository scans?
- Does Box-aware review make failed attempts easier to diagnose than ordinary PR + issue history?
- Does a Box graph become a better planning surface than a flat issue list for the chosen task class?

Only if these behaviors persist should the interface become a truly custom Box-native IDE. Otherwise, the Box layer can remain an extension over established editors.

08 | PRODUCT + RESEARCH

Stage 4 research: Restricted proprietary Boxes

This is the point where “context selection” changes from convenience to access control. The 35-page research proposal becomes a direct technical guide: separate private planning, release authority, external work and private acceptance; treat every boundary crossing as an approved artifact or constrained outcome [R1, pp. 5, 10-16].

Reusable mechanisms

Need	Candidate foundation	Limit that must stay visible
Dependency/context analysis	CodeQL data-flow and program analysis [W11].	Dynamic loading, reflection, library models and business context can be incomplete.
Service interface fidelity	Pact consumer/provider contract testing [W12].	Covers declared interactions, not all business behavior.
Pinned environment	Dev Containers / hermetic builds [W4].	Reproducibility is not confidentiality by itself.
Untrusted execution	Wasmtime/WASI capability sandbox for supported components [W13].	Host configuration/runtime remain trusted; not all software compiles to Wasm.
Agent flow policy	AgentFlow-style labeled flows and task-scoped capabilities [W18].	2026 preliminary research; guarantees are scoped to modeled flows.
Private acceptance	Disposable isolated evaluator with controlled outputs [R1].	Pass/fail, timing and payment can themselves leak information.

Security claim V4 is willing to make

Operational controls can enforce an explicit allowlist of files, interfaces and capabilities for a Box. Stronger semantic confidentiality requires a defined threat model, release policy and evidence. The research proposal explicitly rejects “zero disclosure” and treats semantic leakage plus interactive feedback as open research [R1, pp. 10, 12, 28-29].

OPEN RESEARCH: hiding files is not the same as hiding meaning. Restricted Boxes can ship before universal semantic confidentiality, but the product must label its assurance level honestly.

09 | PRODUCT + RESEARCH

Stage 5 research: Semantically transformed Boxes

The advanced vision is not “obfuscate the whole repository.” It is to retain sensitive responsibility privately while exposing a task-equivalent contract or representation for selected classes: extraction, parameterization, simulators, private adapters or approved intermediate formats [R1, pp. 8-9].

Research foundations that make this credible - but narrow

Foundation	What it contributes	Why it does not solve the whole problem
Pacti assume-guarantee contracts [W14]	Composition, refinement and quotient operations over modeled contracts.	Requires faithful formal models; initial theory is restricted.
EM-Assist [W15]	LLM proposes refactoring; static/IDE analysis validates and applies selected extraction.	Supports proposer/checker architecture, not confidential task synthesis.
Dafny [W16]	Machine-checkable pre/postconditions, invariants and termination for supported code.	Proof is only as good as the specification and model.
ConVer [W17]	LLM-proposed function contracts refined by compositional verification on C benchmarks.	2026 preprint; benchmark-scoped, not private delegation.
Abstraction refinement / CEGAR [R1]	Use counterexamples to refine a restricted abstraction until obligations hold or reject.	General automatic abstraction of arbitrary apps remains open.
CBMC / bounded checks [R1]	Find assertion/memory failures within explicit bounds.	Bounded result, not universal correctness.

The key transfer property

For a supported task class: prove or independently check both (1) the contribution meets its Box contract and (2) that contract is sufficient for the responsibility it plays in the private system.

Research gate

Start with pure transformations, typed adapters and bounded state machines. Require counterexample implementations, version drift and private composition checks. Reject tasks that cannot be represented without disclosing prohibited rules. A selective compiler that says “no” is acceptable if useful coverage remains measurable.

Stages 6-7: External execution and bounties are routing layers

Externalization only becomes attractive after the Box itself is independently executable. The original research proposal already recommends starting with existing contractors or vetted contributors and adding public bounty-board connectors later [R1, p. 18].

Mode	What changes in the Box	What does not change
Internal engineer	Assignee/permissions.	Intent, behavior, environment, evaluator, Git mapping.
Internal agent	Executor identity + compute policy.	Same Box contract and evaluator.
External expert	Release policy, restricted package, frozen acceptance, transfer mapping.	Same work primitive.
Bounty solver	Reward, reservation, identity, dispute, payment terms.	Still a Box attempt; market is not the primitive.

Why marketplace comes last

A marketplace adds matching, liquidity, fraud, dispute resolution and payment policy before proving that task preparation itself saves effort. That compounds uncertainty. The research programme instead measures preparation cost, no-clarification completion, private transfer success and total internal effort before scaling external channels [R1, pp. 18-21].

Independent readiness gate before external execution

- Freeze Box version, environment, evaluator and acceptance categories.
- Run a cold-start solve with an executor that has only released context.
- Validate the result against independent/private checks, not only solver-visible tests.
- Classify missing information as a package defect, not worker failure.
- Track preparation + review + integration + rework cost, not just reward/implementation cost.

Bounty = Box + external routing + payment policy. It is intentionally orthogonal to the core Box platform.

11 | PRODUCT + RESEARCH

Evidence matrix: what is solid, what is plausible, what is open

Claim	Status	Evidence / tool	V4 interpretation
Git-backed isolated Box attempts	ENGINEERING FOUNDATION	Git worktree [W2]	Straightforward MVP substrate.
Reduced visible repository context	ENGINEERING FOUNDATION	Git sparse-checkout [W3]	Convenience/performance only; not security.
Reproducible Box environments	ENGINEERING FOUNDATION	Dev Containers; SWE-bench harness [W4][W5]	Environment fidelity still task-specific.
Browser/remote execution workbench	ENGINEERING FOUNDATION	Coder; VS Code Remote [W9][W10]	Product UX still needs Box-specific design.
Context-rich handoff	RESEARCH PRECEDENT	Codeon; CrowdMicroservices [W7] [W8]	Prior studies are narrower than Box-native development.
Conversational intent discovery	OPEN RESEARCH	ReqElicitGym [W6]	Current models miss implicit requirements; approval required.
Agent/human same work object	PRODUCT DESIGN	Shared Box protocol over same workspace/evaluator	Must be measured across executor types.
Relevant-context analysis	ENGINEERING + RESEARCH	CodeQL; EM-Assist [W11][W15]	Static analysis incomplete; boundary selection remains hard.
Interface substitution	ENGINEERING FOUNDATION	Pact [W12]	Does not cover all state/business semantics.
Restricted execution	ENGINEERING FOUNDATION	Wasmtime/WASI + containers [W13]	Sandboxing != semantic confidentiality.
Derived local obligations	RESEARCH PRECEDENT	Pacti; Dafny; ConVer [W14][W16] [W17]	Useful only for modeled/supported classes.
Safe semantic release	OPEN RESEARCH	Research proposal + information-flow work [R1]	No universal guarantee claimed.
External independent completion	RESEARCH HYPOTHESIS	Proposal pilot design [R1]	Must measure coverage, failures and economics.
Bounty marketplace	KNOWN BUSINESS PATTERN	Historical crowdsourcing precedent in [R1]	Not evidence that Box preparation works.

The end state is ambitious, but the first three stages are buildable largely from existing source-control, workspace and evaluation primitives. The major research risk begins when we require automatic intent completeness, confidentiality, semantic transformation and private-transfer guarantees.

Reference implementation architecture

This is a practical architecture for the first product, not a commitment to specific vendors. The principle is “build the Box semantics; reuse generic infrastructure.”

Component	Responsibility	Suggested MVP mechanism
Box service	Box schema, versioning, status, dependencies, assignments, attempts.	Postgres + service/API; immutable Box version snapshots.
Intent service	Conversation, evidence links, behavior draft, approval record.	LLM-assisted but approval-gated; store sources/decisions.
Repo adapter	Read repo state, create attempts, collect diffs, merge accepted work.	Native Git CLI/lib; worktrees; optional sparse-checkout.
Context analyzer	Candidate files/symbols/dependencies.	Language server + imports + test traces; CodeQL where useful.
Workspace manager	Create isolated runnable Box environment.	Dev Container + Docker; Coder/remote workspace later.
Workbench	Behavior pane, code editor, terminal, evaluator, agent, diff.	Existing web/remote editor shell with Box extension.
Evaluator service	Run published checks and record evidence.	Task-specific command graph in isolated container; SWE-bench-inspired harness.
Agent executor	Consume Box and produce attempt.	Headless process with same workspace/evaluator API.
Review/merge service	Human review, drift handling, merge queue.	Normal Git review semantics + Box metadata.
Provenance store	Box -> attempt -> evaluator -> artifact -> commit chain.	Platform DB + hashes; Git trailer; SLSA-like vocabulary where useful.

A minimal Box manifest

box_id, version, repo_id, base_commit, intent_record, expected_behavior, context_manifest, editable_surface, environment_ref, evaluator_ref, attempt_policy, executor_policy, merge_target, artifact_hashes. Proprietary stages later add disclosure_policy, release_approval, private_mapping and feedback_policy.

This architecture can produce a real product before solving semantic abstraction. The first wedge is a better work object and execution loop.

13 | PRODUCT + RESEARCH

A 12-week product + research programme

The objective is not to finish the end state in 12 weeks. It is to falsify the riskiest early assumptions cheaply while producing a usable Open Box workflow.

Window	Build	Research / evidence	Exit condition
Weeks 1-2	Box schema, Git base mapping, worktree attempts, simple command evaluator.	Select one repo + one task class; manually create 5 Boxes.	Reproducible create -> attempt -> evaluate -> diff cycle.
Weeks 3-4	Intent conversation + behavior approval; simple Box Web list/detail.	Compare AI-assisted vs manual Box preparation; log missing decisions.	Requesters can approve behavior before code exists.
Weeks 5-6	Dev Container / workspace creation; browser or existing-editor workbench.	Cold-start human + agent solves using same Box.	Both executor types can complete at least a subset without repo orientation.
Weeks 7-8	Context suggestions, attempt history, diff review, merge flow.	A/B full repo vs suggested context; measure setup time and failures.	Context reduction helps without blocking completion.
Weeks 9-10	Team assignment, Box dependencies, agent button, evaluator evidence view.	Run 15-30 representative tasks; track rejected tasks too.	Enough signal on usage, no-missing-context and integration.
Weeks 11-12	Hardening + metrics report + restricted-box spike.	Try one narrow restricted adapter/interface case with CodeQL/Pact/sandboxing.	Decision: deepen team product, or begin Stage 4 research; no marketplace yet.

Metrics that matter

- Box preparation time and requester approval time.
- No-missing-context completion, split by human and agent executor.
- Evaluator false confidence: passes that fail independent review/broader tests.
- Integration/merge success and rework.
- Context size vs completion quality.
- User preference and repeat use.
- Coverage: attempted, rejected, failed preparation and completed Boxes - never report only successful cases.

Stage-gate discipline: do not move to proprietary transformation or marketplaces because the architecture diagram looks compelling. Move when the previous rung produces measured value.

14 | PRODUCT + RESEARCH

Canonical thesis and research boundaries

What V4 now claims with confidence

- A Box can be implemented as a deterministic versioned object on top of Git.
- Existing Git, container, remote-workspace and evaluator infrastructure is sufficient to build the early product without inventing a new VCS, sandbox, or IDE engine.
- Humans and agents can be represented as executor choices over the same Box contract and workspace model.
- A web control plane plus execution workbench is a coherent product separation, with a dedicated Box IDE emerging only if usage justifies it.

What V4 treats as empirical hypotheses

- Users will prefer Box-first work to repo-first work for a meaningful task population.
- AI-assisted intent compilation can reduce preparation effort without increasing requirement errors.
- A system can select useful context reliably enough to reduce cognitive/token load without harming completion.
- The same Box contract is efficient for both humans and autonomous coding agents.

What V4 treats as advanced research

- Automatically creating proprietary boundaries that preserve enough semantics for independent completion.
- Proving or tightly bounding information released through task packages and interactive evaluation.
- Deriving local contracts whose satisfaction reliably implies private-system behavior for broad task classes.
- Measuring whether external execution remains economically useful after preparation, review and integration cost.

Canonical thesis: Git remains the durable substrate. The Box becomes the unit of intent, execution, evaluation and coordination. The web is where the Box ecosystem lives; the workbench/IDE is where a Box gets solved. The roadmap begins with ordinary repositories and earns its way toward restricted, transformed and externally executable Boxes.

How to use the two canonical documents

Product Vision Supplement V4 = what to build, in what order, and which claims are already grounded. Private Engineering Delegation research proposal = the deeper technical research programme for contracts, controlled disclosure, verification, transfer and proprietary execution [R1].

Selected evidence and implementation sources

The sources below are intentionally mixed: primary academic work for research claims, official documentation for engineering mechanisms, and the 35-page companion research proposal for the larger evidence review. “Exists” does not mean “solves our whole product.”

[R1] Private Engineering Delegation - Research Proposal, 2 Oct 2026. 35-page companion document; 45-source review covering intent, decomposition, contracts, security, transfer, economics and experiments. *companion file*

[W2] Git. git-worktree documentation - multiple working trees attached to one repository. source

[W3] Git. sparse-checkout documentation - working with a subset of tracked files. source

[W4] Development Containers. Open Dev Container Specification. source

[W5] SWE-bench. Evaluation Harness Reference - Docker-based reproducible patch evaluation. source

[W6] Jin et al. ReqElicitGym: An Evaluation Environment for Interview Competence in Conversational Requirements Elicitation. 2026. source

[W7] Chen et al. Codeon: On-Demand Software Development Assistance. CHI 2017. source

[W8] Aghayi et al. Crowdsourced Behavior-Driven Development: Implementing Microservices through Microtasks. 2019/2020. source

[W9] Coder documentation - self-hosted developer and coding-agent workspaces, IDE integrations. source

[W10] Visual Studio Code. Remote Development documentation. source

[W11] CodeQL. About data flow analysis. source

[W12] Pact. How Pact works - consumer/provider contract testing. source

[W13] Wasmtime. Security - WebAssembly sandboxing and capability-based WASI filesystem access. source

[W14] Incer et al. Pacti: Scaling Assume-Guarantee Reasoning for System Analysis and Design. 2023. source

[W15] Pomian et al. EM-Assist: Safe Automated ExtractMethod Refactoring with LLMs. 2024. source

[W16] Dafny Reference Manual - specification constructs and static verification. source

[W17] Pirzada et al. ConVer: Using Contracts and Loop Invariant Synthesis for Scalable Formal Software Verification. 2026. source

[W18] Shivakumar, Priya, Gao. AgentFlow: A Flow-Centric Policy Language and Framework for Securing LLM Agent Systems. 2026. source

[W19] SLSA Specification v1.2 - provenance and supply-chain integrity vocabulary. source

[W20] Git. interpret-trailers documentation - structured key/value metadata in commit messages. source

Evidence discipline

Academic and benchmark results are cited for the narrow findings they report. Vendor documentation is cited only to establish current advertised mechanisms, not independent effectiveness. V4 does not claim that any source validates the full Box-native system. The full companion proposal contains the larger reference library and the proposed experimental programme.